ACESSO REMOTO: SSH, SCP E RSYNC

Um servidor ficou exposto com SSH na porta 22 e senha habilitada. No primeiro dia, `journalctl -u ssh | grep -c "Failed password"` *devolveu* ***41027*** *— centenas de endereços testando* `root`, `admin`, `git`. *Nenhuma acertou, mas a conta é simples:* ***bastava uma****.*

081
GUIAS LINUX

O SEGREDO QUE TRAFEGA × O SEGREDO QUE FICA PARADO

AUTENTICAÇÃO POR SENHA

CLIENTE → a senha atravessa o canal cifrado → SERVIDOR (compara com o hash)

Qualquer um pode tentar adivinhar, indefinidamente.

AUTENTICAÇÃO POR CHAVE PÚBLICA

CLIENTE (`~/.ssh/id_ed25519` — **privada — nunca sai daqui**)

← 1. desafio aleatório

2. assinatura do desafio →

SERVIDOR (`~/.ssh/authorized_keys` — pública — pode ser divulgada)

3. O servidor verifica a assinatura com a chave pública. Confere? Entra.

Por que isso encerra a força bruta: não há segredo compartilhado para adivinhar — o atacante teria de **forjar uma assinatura** sem a chave privada. **Antes disso vem o TOFU:** na 1ª conexão o SSH mostra a impressão digital e a grava em `known_hosts`; digitar `yes` sem conferir por outro canal (`ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`) abre a janela do ataque de intermediário.

OS TRÊS TÚNEIS: `-L` PUXA, `-R` EMPURRA, `-D` ABRE A REDE

-L local: trago um serviço remoto para cá `ssh -L 8080:127.0.0.1:80 servidor`	sua máquina :8080	túnel SSH	servidor SSH	conexão local	serviço 127.0.0.1:80
-R remoto: exponho um serviço meu lá `ssh -R 9000:127.0.0.1:3000 servidor`	serviço local 127.0.0.1:3000	túnel SSH	servidor SSH :9000	conecta na 9000	quem usa o servidor
-D dinâmico: proxy SOCKS pela rede do servidor `ssh -D 1080 servidor`	navegador SOCKS :1080	túnel SSH	servidor SSH	sai pelo servidor	qualquer destino

A porta que você digita primeiro é sempre a que passa a existir; o `host:porta` seguinte é resolvido **na perspectiva do outro lado** — daí `127.0.0.1:80` em `-L` ser o loopback *do servidor*, não o seu. O `-R` atravessa NAT e firewall de entrada e por padrão só escuta no loopback remoto: `GatewayPorts yes` abre para a rede — e aí você publicou um serviço interno. Túnel sem sessão: `ssh -fN -L …`.

NA PRÁTICA: CHAVE, CONFIG, ENDURECIMENTO E SINCRONIZAÇÃO

```
kali@kali: ~
$ ssh-keygen -t ed25519 -C "vitor@notebook-2026"
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub vitor@servidor
# ele ajusta a permissão — causa nº1 de "copiei e ainda pede
# senha": o sshd recusa em SILÊNCIO; o motivo sai em `ssh -vvv`
$ cat ~/.ssh/config
Host lab
    HostName 10.0.0.50
    IdentityFile ~/.ssh/id_ed25519_lab
    IdentitiesOnly yes    # ofereça SÓ esta chave
Host interno
    ProxyJump lab         # substituto seguro do -A
$ sudo sshd -t && sudo systemctl reload ssh
# PasswordAuthentication no + PermitRootLogin no = 41027 → 0
```

RSYNC: AS OPÇÕES DE TODO COMANDO

`-a`	modo arquivo: recursivo, preserva permissão, dono, tempo e links
`-z` · `-P`	comprime · barra de progresso e **retomada** do interrompido
`-n`	**ensaio**: mostra o que faria, sem fazer
`--delete`	apaga no destino o que sumiu da origem
`--link-dest`	backup incremental com hard links: cada dia parece cópia completa, só o novo ocupa espaço

POR QUE RSYNC E NÃO SCP

diferença	transfere **só o que mudou**: 10 GB com 1 MB alterado movem ~1 MB
scp	aposentado (CVE-2018-20685); sem os `:` copia local→local **em silêncio**

A barra final na origem muda tudo: `rsync -a /origem/ /destino/` copia o **conteúdo**; sem a barra, cria `/destino/origem/`. Com `--delete`, errar a barra **apaga o conteúdo legítimo do destino** — rode com `-n` antes. E **nunca feche a sessão atual depois de mexer no** `sshd_config`: o `sshd -t` pega sintaxe, não lógica — um `AllowUsers` com typo tranca você para fora.

RESUMO RÁPIDO

- ✓ **A chave privada nunca trafega:** o servidor manda um desafio, o cliente assina, o servidor confere com a pública.
- ✓ **Permissão errada = "ainda pede senha":** `700` no `~/.ssh`, `600` na chave e no `authorized_keys`. A recusa é silenciosa.
- ✓ `IdentitiesOnly yes` evita a desconexão por excesso de tentativas quando o agente oferece todas as chaves.
- ✓ `ProxyJump` **substitui o** `-A`: quem tem root no salto se apropria do agente encaminhado.
- ✓ **Trocar a porta 22 é conforto de log, não segurança;** o que protege é chave em vez de senha.

NO KALI

O `ssh` **do Kali vem desabilitado de propósito.** Habilitá-lo numa máquina que vive em Wi-Fi público, sem antes desligar a senha, é o cenário da abertura. E em imagens antigas as **chaves de host vinham idênticas em toda instalação**: `rm /etc/ssh/ssh_host_* && dpkg-reconfigure openssh-server` é passo obrigatório em imagem que você não gerou. `ProxyJump` **é a forma limpa de trabalhar em rede segmentada:** com um salto acordado em contrato, `ssh -J salto interno` chega à rede interna **sem deixar chave nem agente no intermediário.** `-D 1080` + `proxychains` **é o pivô mais simples que existe** — e o ponto em que "alcancei um host" vira "alcancei a rede dele": atravessar para sub-rede fora do escopo ultrapassa a autorização (Lei 12.737/2012). **Chave privada é o segredo mais valioso da máquina:** use *passphrase*, prefira `ssh-add -t`, e trate `authorized_keys` como inventário — quase sempre há uma chave que devia ter saído há meses. `rsync` **é a exfiltração mais discreta que existe**, na porta 22, no meio do tráfego administrativo. **RHEL:** o serviço é `sshd` e trocar a porta exige `semanage port` (SELinux).